Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Спецификация API v1

Протокол, ошибки, права доступа · версия 0.5.0

28 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Термины 4

3 Общие правила протокола 5

4 Аутентификация 6

5 Жизненный цикл заявки 8

6 Аналитика и подбор дат 10

7 Импорт сотрудников 11

8 Списки для руководителя 12

9 Письма 13

10 Живые обновления 14

11 Каталог операций 15

12 Каталог ошибок 22

13 Матрица прав 23

14 Версионирование и совместимость 25

15 Чек-лист новой операции 26

1 О документе

Документ задаёт протокол взаимодействия клиента и сервера «Планировщика отпусков»: общие правила, аутентификацию, семантику операций, права доступа, каталог ошибок и правила совместимости. Формальный контракт — `contracts/openapi-vacation-api-v1.yaml` (OpenAPI 3.1). Документ дополняет его тем, что нельзя выразить схемой: права, переходы состояний, побочные эффекты.

Главное правило: сервер не доверяет клиенту. Любое ограничение, которое клиент проверяет для удобства, сервер проверяет повторно как обязательное.

Соответствие контракта и реализации проверяет тест `server/test/contract.e2e.test.ts`: каждый маршрут сервера описан в контракте, и каждая операция контракта реализована. Каталоги операций и ошибок ниже сформированы из контракта и кода (`tools/docs/gen_tables.py`).

2 Термины

ТЕРМИН	ЗНАЧЕНИЕ
Автор	Сотрудник, на которого оформлена заявка
Согласующий	Непосредственный руководитель автора (поле `managerId`) либо администратор
Кадровая служба	Сотрудник с признаком `isHr`: ведёт сотрудников, импорт, оргструктуру и календарь, видит выгрузки, аналитику и аудит всей организации
Норма	`plannedDays + carryOver` — дни отпуска на год с учётом переноса
Дни отпуска	Календарные дни периода без нерабочих праздничных (ТК РФ ст. 120)
Доступно	Норма − согласованные дни − дни на согласовании
Перенос	Изменение дат согласованного отпуска; прежние даты хранятся в `changeFrom` до решения
Запрос удаления	Согласованный отпуск с `pendingDeletion = true`, ожидающий решения руководителя
Порог конфликта	Доля отдела в отпуске (настройка организации, по умолчанию 50 %), начиная с которой день конфликтный

3 Общие правила протокола

1. Базовый префикс `/api/v1`, тело — JSON, кодировка UTF-8; даты — `YYYY-MM-DD`, метки времени — RFC 3339 в UTC.
2. Аутентификация — заголовок `Authorization: Bearer <access JWT>`; refresh-токен передаётся только httpOnly-cookie `vp_refresh`.
3. Пагинация списков: `page` (с 1) и `per_page` (1–200, по умолчанию 50); ответ `{ items, meta: { page, perPage, total } }`.
4. Частичное изменение — `PATCH` только с изменяемыми полями. Неизвестные поля запроса отбрасываются, клиент обязан игнорировать неизвестные поля ответа.
5. Повторное решение по заявке не дублирует эффект: если заявка уже не на согласовании, сервер отвечает `409 stale-state`, клиент перечитывает данные.
6. Тяжёлые чтения ограничены: график — не длиннее 370 дней, график работы и подбор дат — не длиннее 400 дней.
7. Ограничение частоты входа: 10 попыток в минуту на пару «адрес + логин» и 60 на адрес → `429 rate-limited` с заголовком `Retry-After` (секунд). Счётчики общие для всех процессов и экземпляров сервера.
8. Ошибки — только `application/problem+json` (RFC 9457) с кодом из каталога ниже.
9. Перегрузка или недоступность БД — `503 service-unavailable` с заголовком `Retry-After`; любую операцию чтения клиент может повторить по истечении паузы.
10. `GET /schedule` отдаёт слабый `ETag` по версии данных и `Cache-Control: private, no-cache`. Запрос с `If-None-Match`, равным текущему `ETag`, получает `304 Not Modified` без тела. Версия меняется при любом изменении заявок, сотрудников, отделов, проектов, графиков работы и правил организации.

4 Аутентификация

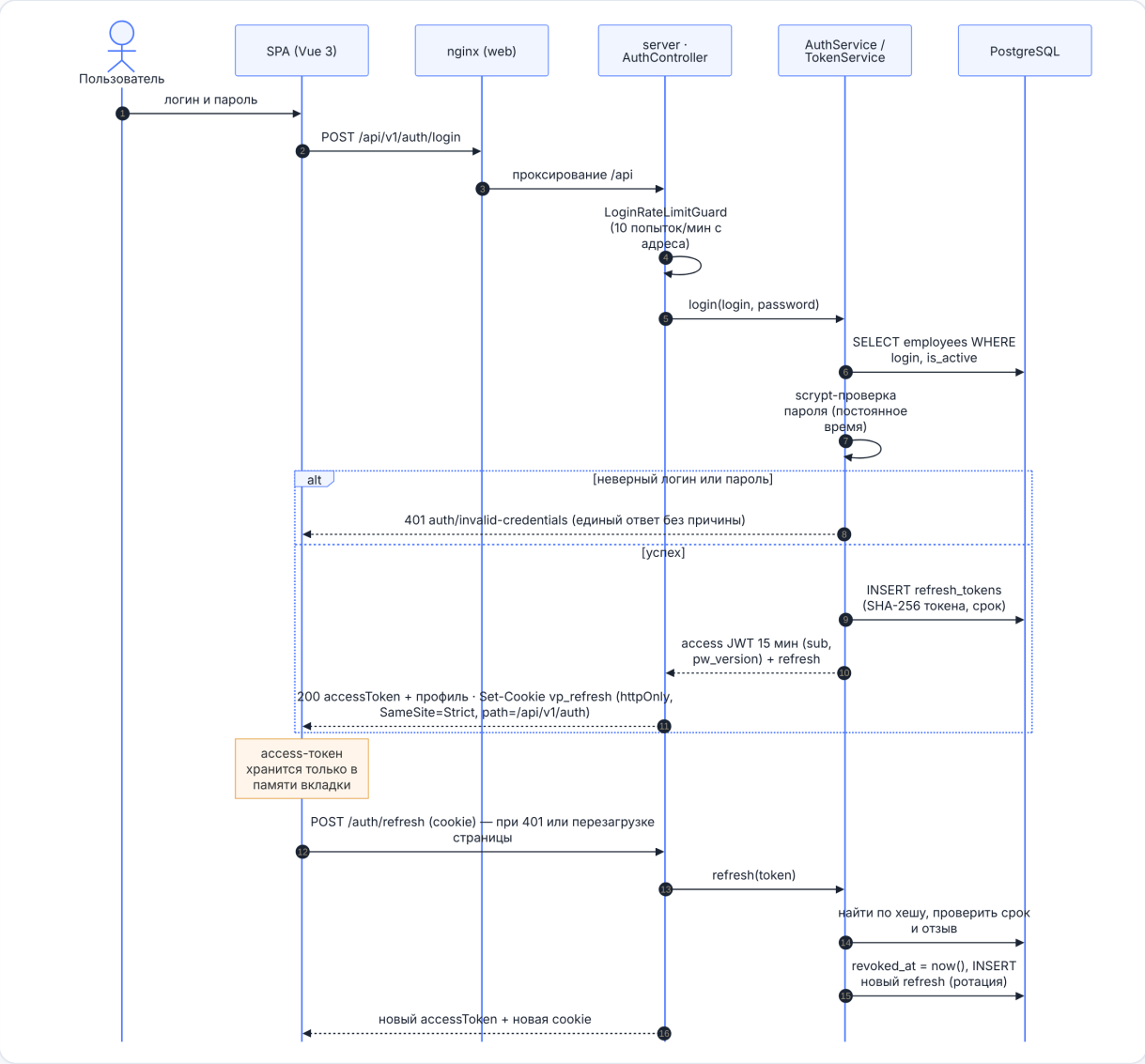


Рис. 1. Вход и продление сессии

ОПЕРАЦИЯ	ПОВЕДЕНИЕ
`POST /auth/login`	Логин и пароль → `accessToken` (15 минут) и профиль; refresh-cookie на 30 дней (`HttpOnly` , `SameSite=Strict` , `Secure` в production, `Path=/api/v1/auth`)
`POST /auth/refresh`	По cookie выдаёт новый access-токен и новую cookie; использованный refresh-токен отзывается (ротация)
`POST /auth/logout`	Отзывает refresh-токен и удаляет cookie

ОПЕРАЦИЯ	ПОВЕДЕНИЕ
`GET /auth/me`	Профиль и вычисленные права: подчинённые, администратор
`POST /auth/password`	Смена своего пароля по старому; все сессии завершаются
`PATCH /auth/me`	Личные настройки: `emailNotifications` — письма о заявках и сводка
`GET /auth/oidc/config`	Включён ли единый вход, подпись кнопки, разрешён ли вход по паролю
`GET /auth/oidc/start` → `GET /auth/oidc/callback`	Единый вход OpenID Connect: authorization code + PKCE; успех — refresh-cookie и переход в приложение, отказ — `/login?sso=cancelled|expired|invalid|unknown`

Смена пароля и увольнение сотрудника отзывают все выданные токены: следующий запрос со старым access-токеном получает `401 auth/token-expired` (в других процессах сервера — не позже чем через 10 секунд), refresh-токены отзываются сразу. `auth/token-expired` клиент обрабатывает как истёкший сеанс: пробует `refresh`, при неудаче — экран входа.

При `LOCAL_LOGIN=false` вход по паролю сотрудникам запрещён (`403 auth/forbidden`), администраторам он остаётся как аварийный.

Неверный логин или пароль — единый ответ `401 auth/invalid-credentials`, который не раскрывает, что именно неверно. Клиент при `401` один раз вызывает `refresh` (общий для всех параллельных запросов) и повторяет запрос; при неудаче — переход на экран входа.

5 Жизненный цикл заявки

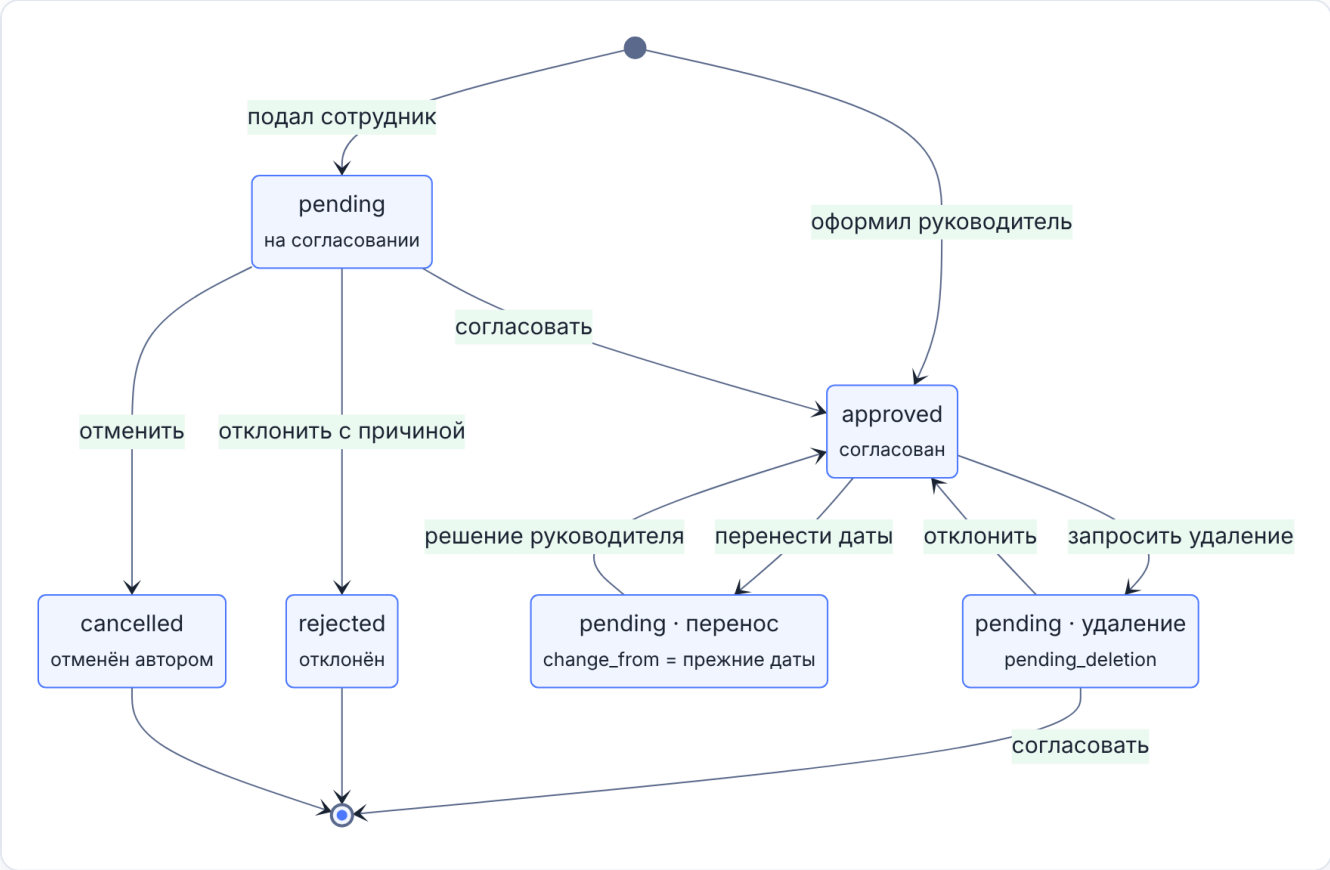


Рис. 2. Состояния заявки

1. `POST /vacation-requests` создаёт заявку `pending`. Если её оформляет руководитель за подчинённого или администратор — сразу `approved`. Сервер проверяет права, формат и порядок дат, «не в прошлом» (кроме администратора), пересечения с заявками сотрудника (`409 overlapping-request`) и лимит дней при жёсткой политике (`409 limit-exceeded`).
2. Автор правит даты `pending`-заявки через `PATCH`. Для `approved` тот же `PATCH` создаёт перенос: заявка переходит в `pending`, прежние даты сохраняются в `changeFrom`.
3. `DELETE` отменяет `pending`-заявку (`cancelled`) или запрашивает удаление `approved` (`pending` + `pendingDeletion`).
4. Согласующий вызывает `POST …/approve`: новая заявка и перенос становятся `approved`, запрос удаления удаляет заявку (событие остаётся в истории и аудите).
5. `POST …/reject` с обязательной причиной (3–1000 символов): новая заявка — `rejected`; перенос — возврат к прежним датам и `approved`; запрос удаления — отпуск остаётся `approved`.
6. Каждый переход пишет событие в историю заявки, запись `audit_events` и уведомление — в одной транзакции.

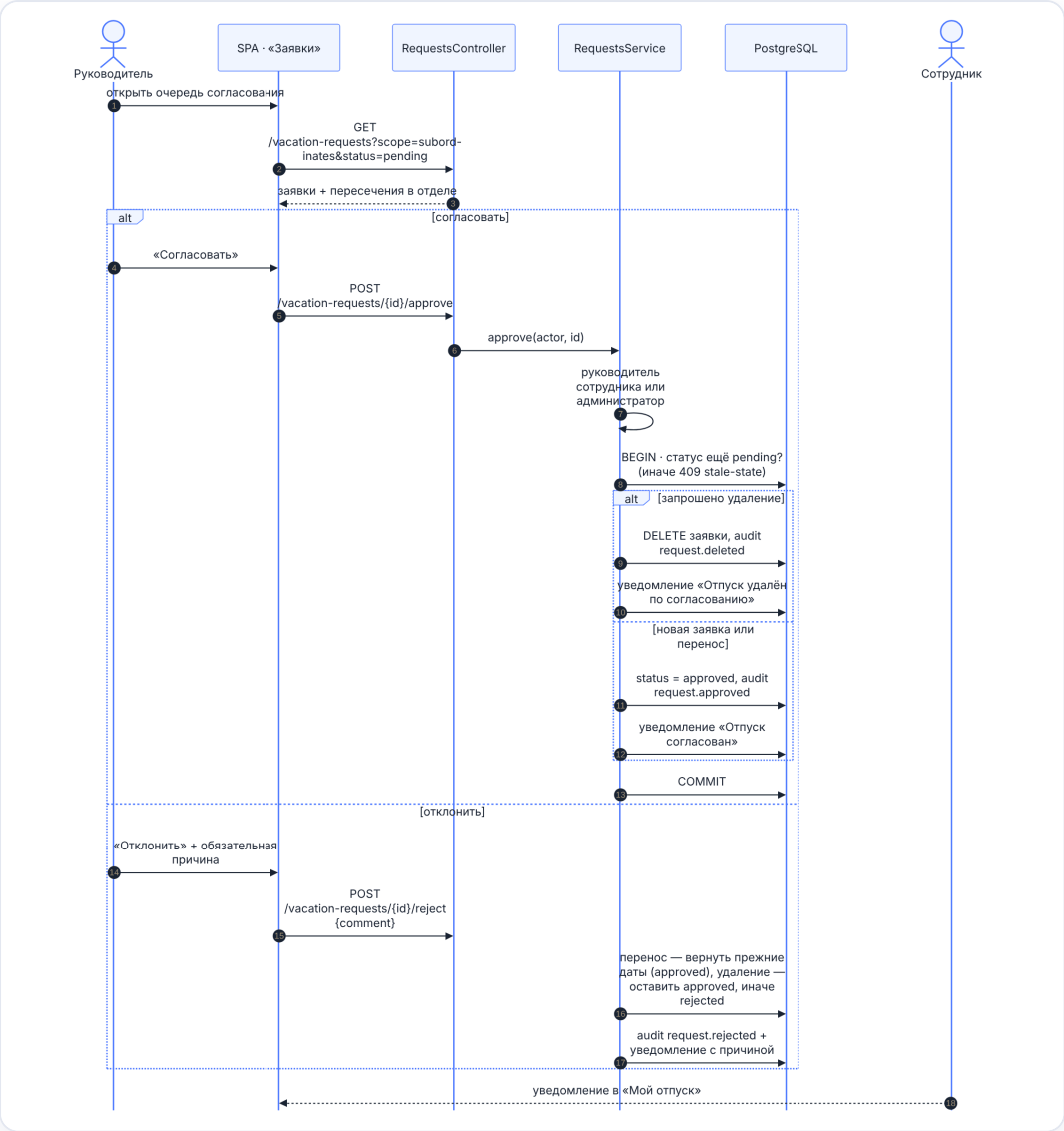


Рис. 3. Решение руководителя

6 Аналитика и подбор дат

ОПЕРАЦИЯ	ПОВЕДЕНИЕ
`GET /analytics/departments`	Отделы, доступные пользователю: администратору — все; руководителю — отделы, где работают его подчинённые или которыми он руководит; сотруднику — свой
`GET /analytics/overview?dept&year`	Доля отдела в отпуске по месяцам (только рабочие дни; отдельно с учётом заявок на согласовании), периоды конфликтов с составом, распределение дней по сотрудникам, выводы с уровнем важности
`GET /analytics/team?dept|project&date`	Состав на дату: офис, удалёнка, отпуск (с датой окончания), выходной; следующий отпуск, нераспределённые дни, часть в 14 дней, заявки на согласовании
`GET /analytics/suggestions?days&from&to&employee`	До пяти непересекающихся окон для отпуска заданной длины с оценкой и объяснением; праздники продлевают окно, окна с превышением порога и пересечением с собственными отпусками исключаются

Подбор дат доступен для себя, руководителю — для прямых подчинённых, администратору — для любого сотрудника. Статус «офис / удалёнка» вычисляется так: отпуск важнее всего, затем разовое исключение, затем выходной или праздник, затем недельный шаблон (по умолчанию — офис).

7 Импорт сотрудников

`POST /imports/employees?dryRun=true|false` принимает файл XLSX/CSV (multipart, поле `file`, до 2 МБ, до 5000 строк) или JSON `{ rows }` — точку синхронизации с кадровой системой. Колонки: ФИО (обязательно), Логин, Должность, Отдел, Руководитель (логин или ФИО), Норма дней, Перенос, Тип занятости, Email; шаблон — `GET /imports/employees/template`.

ПРАВИЛО	ПОВЕДЕНИЕ
Сопоставление	По логину; без логина — по ФИО среди работающих (несколько совпадений — ошибка строки)
Новый сотрудник	Логин транслитом ФИО с числовым суффиксом при совпадении; одноразовый пароль — в ответе применения один раз
Пустая ячейка	Значение не меняется; повторная загрузка того же файла ничего не меняет
Новые отделы	Создаются; руководитель может быть создан этим же файлом
Ошибки	По строкам в плане; `dryRun=false` при ошибках отклоняется целиком (`400`)
Права	Кадровая служба и администратор; кадровая служба не меняет администраторов и коллег по службе

8 Списки для руководителя

ОПЕРАЦИЯ	ПОВЕДЕНИЕ
`GET /employees?manager={id}`	Только прямые подчинённые сотрудника `id` (с v0.5.0). Экран согласования и график работы руководителя берут подчинённых так, а не перебором первых 200 сотрудников организации
`GET /vacation-requests`	Элементы списка несут `employeeName` и `employeeDept` (с v0.5.0): очередь согласования запрашивает график только отделов из очереди, чтобы показать пересечения
`POST /notifications/read-all`	Отмечает прочитанными все уведомления текущего сотрудника одним запросом, ответ `{ marked }` (с v0.5.0)

Норма дней и перенос в карточке сотрудника (`POST` / `PATCH /employees`) — числа с точностью до сотых, от 0 до 365, как в импорте; до v0.5.0 API принимал только целые, и правка карточки округляла перенос из импорта.

9 Письма

Письма отправляются, если на сервере задан `SMTP_HOST`. Письмо создаётся в той же транзакции, что уведомление (`email_outbox`), и уходит фоном с повторами; сбой почты не влияет на операцию. Руководитель получает письмо о новой заявке, переносе и запросе удаления, сотрудник — о решении. В пятницу после 12:00 по времени организации руководители получают сводку: кто из команды в отпуске на следующей неделе и сколько заявок ждут решения. Отписка — `PATCH /auth/me`. Администратор проверяет SMTP операцией `POST /settings/test-email`.

10 Живые обновления

Клиент держит поток Server-Sent Events и перечитывает данные по сигналу вместо опроса по таймеру.

ШАГ	ПОВЕДЕНИЕ
`POST /events/ticket`	По access-токену выдаёт билет `{ ticket }` на 60 секунд. Билет — JWT с отдельной аудиторией: им нельзя вызвать другие операции
`GET /events?ticket=…`	Открывает поток `text/event-stream`. Билет недействителен, пароль сменён или сотрудник уволен — `401`
`event: hello`	Сразу после открытия: `{ unread, version }` — непрочитанные уведомления и версия данных
`event: notifications`	Новые уведомления сотрудника: `{ unread }`; клиент перечитывает уведомления и очередь заявок
`event: data`	Версия данных изменилась: `{ version }`; не чаще раза в 15 секунд. Клиент тихо перечитывает график на видимой вкладке
`event: revoked`	Сеанс отозван — пароль сменён или сброшен, сотрудник уволен: `{}`, затем сервер закрывает поток. Сервер сверяет подключённых раз в 10 секунд; клиент сразу переходит на экран входа с сообщением «Сеанс завершён»
Комментарий `:` раз в 25 секунд	Держит соединение через прокси; данных не несёт

Сервер проверяет события раз в 2 секунды. При обрыве клиент закрывает поток, получает новый билет и переподключается с нарастающей паузой от 4 до 60 секунд; пока потока нет, он опрашивает уведомления раз в 30 секунд, как до v0.5.0.

11 Каталог операций

Всего операций: **58** в 43 путях; формат — JSON, ошибки — `application/problem+json`.

Вход и сессии (9)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
POST	`/api/v1/auth/login`	Вход по логину и паролю	`login`
POST	`/api/v1/auth/refresh`	Новый access-токен по refresh-cookie (ротация; старый refresh отзывается)	`refresh`
POST	`/api/v1/auth/password`	Смена своего пароля (все сессии завершаются)	`changeOwnPassword`
POST	`/api/v1/auth/logout`	Завершить сессию	`logout`
GET	`/api/v1/auth/me`	Текущий пользователь и его права	`getMe`
PATCH	`/api/v1/auth/me`	Личные настройки — письма о заявках	`updateMyPreferences`
GET	`/api/v1/auth/oidc/config`	Единый вход — включён ли, подпись кнопки, разрешён ли вход по паролю	`getSsoConfig`
GET	`/api/v1/auth/oidc/start`	Начать вход через провайдер OpenID Connect	`startSso`
GET	`/api/v1/auth/oidc/callback`	Возврат от провайдера	`ssoCallback`

Сотрудники (6)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/employees`	Список сотрудников (постранично, с фильтрами)	`listEmployees`
POST	`/api/v1/employees`	Создать сотрудника (только администратор)	`createEmployee`
GET	`/api/v1/employees/{id}`	Карточка сотрудника	`getEmployee`
PATCH	`/api/v1/employees/{id}`	Изменить поля сотрудника	`updateEmployee`
DELETE	`/api/v1/employees/{id}`	Уволить/удалить сотрудника (только администратор)	`deleteEmployee`
PUT	`/api/v1/employees/{id}/password`	Смена пароля (свой — со старым, чужой — только администратор)	`changePassword`

Баланс (1)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/employees/{id}/balance`	Баланс дней отпуска за год	`getBalance`

График и режим работы (5)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
PUT	`/api/v1/employees/{id}/work-pattern`	Недельный шаблон офис/удалёнка (свой или подчинённый)	`setWorkPattern`
PUT	`/api/v1/employees/{id}/work-overrides`	Ручное переопределение режима на дату	`setWorkOverride`
GET	`/api/v1/employees/{id}/work-schedule`	График работы — недельный шаблон и исключения за период (с v0.3.0)	`getWorkSchedule`

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
DELETE	`/api/v1/employees/{id}/work-overrides/{date}`	Снять исключение — на дату снова действует недельный шаблон (с v0.3.0)	`deleteWorkOverride`
GET	`/api/v1/schedule`	График отпусков и режимов за период	`getSchedule`

Заявки на отпуск (7)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/vacation-requests`	Заявки (свои, подчинённых или все — по правам)	`listVacationRequests`
POST	`/api/v1/vacation-requests`	Создать заявку (за себя или за подчинённого)	`createVacationRequest`
GET	`/api/v1/vacation-requests/{id}`	Заявка с историей	`getVacationRequest`
PATCH	`/api/v1/vacation-requests/{id}`	Изменить свою pending-заявку или запросить перенос approved	`updateVacationRequest`
DELETE	`/api/v1/vacation-requests/{id}`	Отменить свою pending-заявку или запросить удаление approved	`cancelVacationRequest`
POST	`/api/v1/vacation-requests/{id}/approve`	Согласовать (руководитель сотрудника или администратор)	`approveVacationRequest`
POST	`/api/v1/vacation-requests/{id}/reject`	Отклонить с обязательным комментарием	`rejectVacationRequest`

Справочники (12)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/departments`	Отделы с руководителями и численностью	`listDepartments`
POST	`/api/v1/departments`	Создать отдел (администратор)	`createDepartment`
PATCH	`/api/v1/departments/{name}`	Переименовать, сменить руководителя	`updateDepartment`
DELETE	`/api/v1/departments/{name}`	Удалить отдел (нельзя при наличии сотрудников)	`deleteDepartment`
GET	`/api/v1/projects`	Проекты	`listProjects`
POST	`/api/v1/projects`	Создать проект (администратор)	`createProject`
PATCH	`/api/v1/projects/{id}`	Изменить проект	`updateProject`
DELETE	`/api/v1/projects/{id}`	Удалить проект (нельзя при наличии участников)	`deleteProject`
GET	`/api/v1/groups`	Пользовательские группы для фильтров	`listGroups`
POST	`/api/v1/groups`	Создать группу	`createGroup`
PATCH	`/api/v1/groups/{id}`	Изменить название или состав	`updateGroup`
DELETE	`/api/v1/groups/{id}`	Удалить группу	`deleteGroup`

Настройки (3)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/settings`	Настройки организации	`getSettings`
PATCH	`/api/v1/settings`	Изменить настройки (только администратор)	`updateSettings`

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
POST	`/api/v1/settings/test-email`	Проверочное письмо администратору (проверка SMTP после развёртывания)	`sendTestEmail`

Производственный календарь (2)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/calendar/holidays/{year}`	Праздники и переносы производственного календаря	`getHolidays`
PUT	`/api/v1/calendar/holidays/{year}`	Загрузить или изменить календарь года (кадровая служба, администратор)	`setHolidays`

Уведомления (3)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/notifications`	Уведомления текущего пользователя	`listNotifications`
POST	`/api/v1/notifications/read-all`	Пометить прочитанными все уведомления (с v0.5.0)	`markAllNotificationsRead`
POST	`/api/v1/notifications/{id}/read`	Пометить прочитанным	`markNotificationRead`

Аудит (1)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/audit-events`	Журнал изменений (администратор)	`listAuditEvents`

imports (2)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
POST	`/api/v1/imports/employees`	Импорт сотрудников и остатков — предпросмотр или применение	`importEmployees`
GET	`/api/v1/imports/employees/template`	Шаблон XLSX для импорта сотрудников	`getImportTemplate`

events (2)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
POST	`/api/v1/events/ticket`	Билет на поток событий (60 секунд)	`getEventsTicket`
GET	`/api/v1/events`	Поток событий (text/event-stream)	`streamEvents`

Отчёты (1)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/reports/vacations`	Выгрузка отпусков за период (XLSX)	`exportVacations`

Аналитика (4)

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/analytics/departments`	Отделы, доступные пользователю для аналитики (с v0.3.0)	`listAnalyticsDepartments`
GET	`/api/v1/analytics/overview`	Нагрузка отдела по месяцам, периоды конфликтов, распределение дней, выводы (с v0.3.0)	`getDepartmentOverview`

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ	ОПЕРАЦИЯ
GET	`/api/v1/analytics/team`	Состав отдела или проекта на дату — офис, удалёнка, отпуск, выходной (с v0.3.0)	`getTeamToday`
GET	`/api/v1/analytics/suggestions`	Лучшие окна для отпуска с учётом нагрузки отдела и праздников (с v0.3.0)	`suggestVacationDates`

12 Каталог ошибок

`type` — `https://vacation.planner/problems/<код>`; `title` — короткий текст для человека, `detail` — уточнение для конкретного случая. Новые коды добавляются одновременно в `server/src/infra/problems.ts`, контракт и документ 5 (тексты интерфейса).

КОД (TYPE)	HTTP	ЗАГОЛОВОК (TITLE)
`auth/invalid-credentials`	401	Неверный логин или пароль
`auth/token-expired`	401	Сессия истекла
`auth/forbidden`	403	Недостаточно прав для этого действия
`validation/invalid-field`	400	Некорректно заполнено поле
`validation/bad-date-range`	400	Дата окончания раньше даты начала или даты в прошлом
`not-found`	404	Запись не существует или удалена
`overlapping-request`	409	Период пересекается с существующей заявкой
`limit-exceeded`	409	Превышен остаток дней отпуска
`stale-state`	409	Данные изменились: обновите страницу и повторите
`last-admin`	409	Нельзя удалить или разжаловать последнего администратора
`department-not-empty`	409	В отделе ещё есть сотрудники
`rate-limited`	429	Слишком много запросов. Повторите позже.
`calendar-missing`	400	Для этого года не загружен производственный календарь
`service-unavailable`	503	Сервис перегружен. Повторите через несколько секунд.

13 Матрица прав

ДЕЙСТВИЕ	СОТРУДНИК	РУКОВОДИТЕЛЬ	КАДРОВАЯ СЛУЖБА	АДМИНИСТРАТОР
Смотреть график организации	да	да	да	да
Подавать, переносить, отменять свои заявки	да	да	да	да
Оформлять отпуск за другого (сразу согласован)	—	прямым подчинённым	—	любому
Согласовывать и отклонять	—	заявки прямых подчинённых	—	все заявки
Видеть баланс и график работы	свои	свои и подчинённых	любые	любые
Менять график работы	свой	свой и подчинённых	свой	любой
Команда и аналитика	свой отдел	свои отделы	все отделы	все отделы
Подбор дат	для себя	для себя и подчинённых	для любого	для любого
Выгрузка XLSX	—	свой отдел, если руководит им, иначе подчинённые	организация	организация
Сотрудники, импорт, сброс паролей	—	—	кроме администраторов и коллег по службе	да
Оргструктура, производственный календарь	—	—	да	да
Журнал аудита	—	—	да	да

ДЕЙСТВИЕ	СОТРУДНИК	РУКОВОДИТЕЛЬ	КАДРОВАЯ СЛУЖБА	АДМИНИСТРАТОР
Правила организации, проверочное письмо	—	—	—	да
Выдача ролей «администратор» и «кадровая служба»	—	—	—	да

Права руководителя и кадровой службы складываются: руководитель отдела кадров согласует заявки своих подчинённых.

14 Версионирование и совместимость

1. Версия API — в префиксе пути. Несовместимое изменение — новый префикс `/api/v2`, обе версии работают не меньше 6 месяцев.
2. Добавляющие изменения (новые операции, необязательные поля, значения перечислений в конце) делаются в рамках v1 и отмечаются в описании операции «с vX.Y.Z».
3. Изменения `contracts/` проходят ревью владельца (CODEOWNERS); дифф контракта обязателен в описании PR.
4. Тест сверки контракта с реализацией входит в CI и блокирует слияние при расхождении.

15 Чек-лист новой операции

- Операция описана в контракте: `operationId`, параметры, схемы ответа, коды ошибок.
- DTO с class-validator: границы длин и чисел, форматы дат.
- Права проверены: роль — guard, объект — сервис.
- Ошибки — только из каталога.
- Изменение данных, история, аудит и уведомления — в одной транзакции.
- Тесты: успех, каждая ошибка, отказ в правах; для списков — пагинация.
- Обновлены тексты интерфейса (документ 5), CHANGELOG и таблицы документов (`tools/docs/gen_tables.py`).